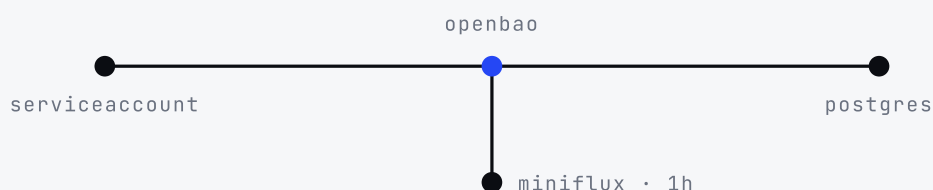


DYNAMIC DATABASE CREDENTIALS

Short-lived PostgreSQL access with OpenBao and the External Secrets Operator on Kubernetes.



Dynamic Database Credentials with OpenBao

Connecting PostgreSQL through the Database Secrets Engine and the External Secrets Operator – Miniflux on Kubernetes as the worked example

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI
- A word about the passwords in this guide
- License and liability
- Who this is for
- The building blocks
- The architecture in one picture

Part I – The cluster

- OpenBao in the cluster
- PostgreSQL via Helm
- External Secrets Operator via Helm
- Namespace and ServiceAccount of the application

Part II – Setting up OpenBao

- Kubernetes auth: who may log in?
- Database secrets engine: the connection
- Database secrets engine: the roles
- Policy: what may the logged-in token do?

Part III – Preparing PostgreSQL

- The role OpenBao works with
- The owner role for Miniflux
- Verification: does the session really run as `miniflux`?

Part IV – Connecting the External Secrets Operator

- SecretStore: how ESO reaches OpenBao
- VaultDynamicSecret: the generator
- ExternalSecret: the target secret
- Apply and check

Part V – The application

- The Deployment
- Looking at the UI

Part VI – What went wrong, and why

- permission denied for schema public
- "miniflux" is not an allowed role
- permission denied (403) on everything
- permission denied to grant role "miniflux"
- Logs of the wrong pod

Part VII – Operation

- The TTL trap: environment variables are static

Retiring the root token

Infrastructure as Code

Plain-text passwords

Lease hygiene

Appendix A – All commands at a glance

Appendix B – Glossary

What this is about

An application needs access to a database. The classic way: a user with a fixed password that lives somewhere in a Kubernetes Secret – often for months or years, copied into Helm values, CI variables and notebooks. If it leaks, nobody notices, and rotating it is so tedious that it never happens.

This guide describes the alternative: **the application no longer gets a fixed password at all**. Instead, OpenBao creates a database user with a short lifetime on demand (here: one hour), the External Secrets Operator (ESO) writes the credentials into a Kubernetes Secret, and the application reads them from an environment variable as it always has. When the lifetime expires, OpenBao drops the user again; ESO has long since fetched a new one.

The running example is concrete and reproducible: **Miniflux**, a lean RSS reader written in Go that needs a PostgreSQL database and runs its own schema migrations at startup. That one detail – the app creates its own tables – is what makes the example instructive, because it forces us to think about who owns database objects.

Everything in this guide was actually carried out on an RKE2 homelab cluster. The errors that came up along the way are included – they are the most valuable part.

It is the second part of a series. N° 1, *OpenBao on Kubernetes*, builds the OpenBao used here – Helm, init and unseal, Kubernetes auth. N° 3 describes the same OpenBao on a VM, with Ansible, OpenTofu and disaster recovery. This guide assumes N° 1 and repeats from it only what the context needs.

Why by hand, and why without AI

In a real setup you do not build this from the CLI. OpenBao roles, policies, auth methods and database connections belong in HashiCorp Terraform or OpenTofu: versioned, reproducible, reviewable. The appendix names the matching resources, and the repository this guide grew out of already manages auth and policies that way.

Even so, this guide performs every step by hand once – and deliberately without AI assistance as a shortcut. A Terraform module written by someone else, or by a language model, hides exactly what you need to have understood when it breaks at three in the morning: which components are involved, who authenticates to whom, who needs which permissions, and in what order the flow runs.

The yardstick is simple. Whoever has worked through this guide should be able to **draw the architecture from the next section on a blank sheet of paper, unaided**: the four components, the six steps, the two gates (the connection's allowlist and the policy) and the one PostgreSQL role that owns everything. Whoever can do that can also write the Terraform module – or judge whether what an AI proposes is correct.

Tools like [nyrvex](#), which generates the configuration of an AI platform's secret store and identity provider, take these steps off your hands later. You should have walked them yourself once, to be able to judge what was generated.

A word about the passwords in this guide

This guide contains passwords in plain text: postgres123, demo123, admin123, plus the usernames of dynamic database users. That is not an oversight. Everything here comes from a **development environment** – a homelab cluster with no access from outside, no real data, and a database that can be thrown away and rebuilt at any time. The values were chosen so you recognise them while reading, not so they protect anything.

In every other environment the opposite applies: no password in Helm values, none in a deployment manifest, none in a piece of documentation. That is precisely what this guide is for – the application no longer gets a fixed password, and the few static secrets that remain (root credentials for the database, the Miniflux admin password) belong in OpenBao's KV engine and from there into the cluster via an ExternalSecret. Part VII comes back to this.

License and liability

This guide is licensed under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided as is, without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk.

Who this is for

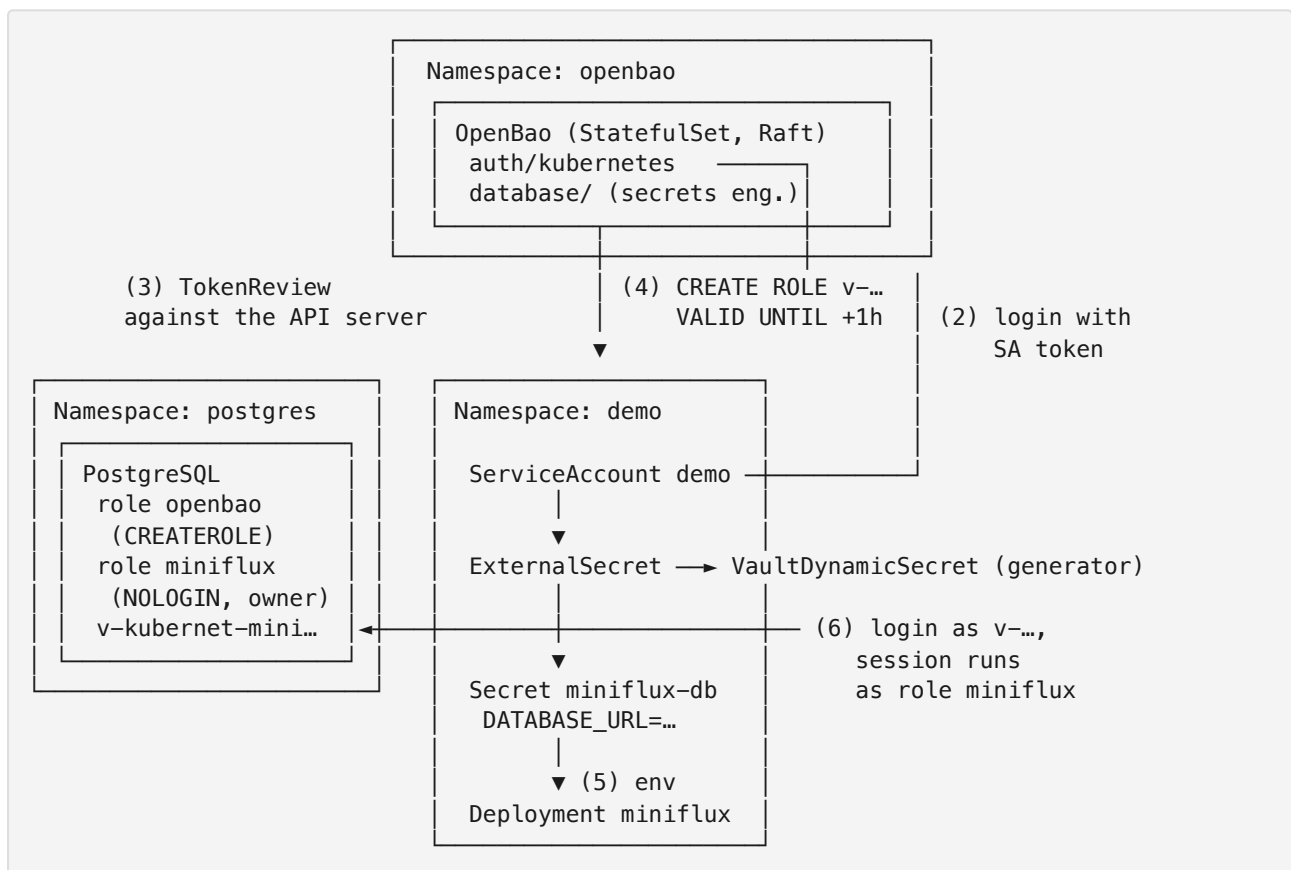
Readers who know Kubernetes basics (Deployment, Secret, ServiceAccount, Helm), can operate PostgreSQL, and have seen OpenBao or HashiCorp Vault before. OpenBao is a fork of Vault; everything here applies to both, only the CLI name (bao instead of vault) and the environment variables (BAO_ADDR instead of VAULT_ADDR) differ.

This guide is free and deliberately short: a single use case, followed through to a running application. It explains what it needs, but it is no substitute for an introduction. If you want to understand auth methods, policies, secrets engines and leases from the ground up – with labs that run on a laptop – that is in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice): 24 chapters, including the database secrets engine and the Kubernetes auth method; OpenBao arrives in Chapter 18.

The building blocks

Component	Version	Role in the setup
RKE2 Kubernetes	–	Runtime, Traefik as ingress
OpenBao (Helm chart openbao/openbao)	chart 0.29.4, app 2.6.2	Creates and revokes database users; authenticates workloads
PostgreSQL (Helm chart bitnami/postgresql)	chart 18.11.3, PG 18.6	The database
External Secrets Operator	2.10.0	Fetches secrets from OpenBao and writes Kubernetes Secrets
Miniflux	2.3.3	The example application

The architecture in one picture



The flow hidden in this picture:

1. ESO reconciles the `ExternalSecret` and triggers the `VaultDynamicSecret` generator.
2. The generator logs in to OpenBao – using the token of the `ServiceAccount demo` (Kubernetes auth method).
3. OpenBao verifies the token via `TokenReview` against the Kubernetes API server and issues an OpenBao token carrying the policy `eso-demo`.

4. With that token the generator reads `database/creds/miniflux`. OpenBao connects to the database as role `openbao` and executes the `creation_statements`: a new login user `v-kubernetes-miniflux-...` is created, valid for one hour.
5. ESO renders a `DATABASE_URL` from it via template and writes it to the Kubernetes Secret `miniflux-db`. The Deployment reads it as an environment variable.
6. Miniflux connects. Thanks to `SET ROLE` the session immediately runs as `miniflux` – every table the migration creates belongs to that fixed role, not to the short-lived user.

The chapters follow this path: first the infrastructure in the cluster, then OpenBao, then PostgreSQL, then ESO, then the application. At the end come the errors that occurred along the way and what to watch out for in operation.

Part I – The cluster

OpenBao in the cluster

OpenBao runs as a StatefulSet in the namespace `openbao`, with Raft storage on a PVC, standalone, initialised and unsealed by hand. How it got there – Helm values, init, unseal, ingress – is the subject of N° 1. For this guide three things matter:

- The in-cluster service is `openbao.openbao.svc:8200`; that is the address ESO uses.
- The Kubernetes auth method is enabled and configured (`auth/kubernetes/config` points at `https://kubernetes.default.svc:443`).
- Setup happens in a shell inside the pod, because `BAO_ADDR=http://127.0.0.1:8200` is already set there:

```
kubectl exec -it -n openbao openbao-0 -- sh
/ $ bao login          # enter the root token
```

All `bao` commands in this guide were run in such a shell. A common trap: a terminal with `BAO_ADDR` pointing at the ingress but without a valid token returns `403 permission denied` for every call – it looks like a policy problem, but it is simply a missing login.

PostgreSQL via Helm

The Bitnami chart, minimally configured (`postgres/values.yaml`):

```
auth:
  postgresPassword: "postgres123"  # superuser – plain text for the demo only
  username: "demo"
  password: "demo123"
  database: "demodb"

primary:
  persistence:
    enabled: true
    size: 10Gi
```

```
helm upgrade --install postgres oci://registry-1.docker.io/bitnamicharts/postgresql \
  --namespace postgres --create-namespace \
  --values postgres/values.yaml
```

Result: a pod `postgres-postgresql-0`, the service `postgres-postgresql.postgres.svc.cluster.local:5432`, database `demodb`.

For admin work the fastest route is directly inside the pod. The Bitnami image stores the superuser password in a file, hence the `cat`:

```
kubectl exec -it -n postgres postgres-postgresql-0 -- sh -c \  
'PGPASSWORD="$(cat /opt/bitnami/postgresql/secrets/postgres-password)" \  
psql -U postgres -d demodb'
```

It is worth putting this in a small shell function or alias; it will be needed a few more times.

External Secrets Operator via Helm

```
helm repo add external-secrets https://charts.external-secrets.io  
helm upgrade --install external-secrets external-secrets/external-secrets \  
--version 2.10.0 \  
--namespace external-secrets --create-namespace
```

ESO ships the CRDs SecretStore, ExternalSecret and – important for this guide – the **generators**, among them VaultDynamicSecret. A generator is an object that *creates* a new secret on every sync instead of reading an existing one. That is exactly what dynamic credentials need.

Namespace and ServiceAccount of the application

```
kubectl create namespace demo  
kubectl create serviceaccount demo -n demo
```

This ServiceAccount is the **identity** ESO uses to log in to OpenBao. It needs no Kubernetes RBAC permissions; OpenBao merely checks that a token for exactly this account in exactly this namespace is presented.

Part II – Setting up OpenBao

All commands in this part run with the root token in the pod shell (see Part I). In a real setup this moves into Terraform/OpenTofu after the initial bootstrap – more on that in Part VII.

Kubernetes auth: who may log in?

The kubernetes auth method lets workloads log in with their ServiceAccount token. OpenBao forwards the token to the API server for verification (TokenReview).

```
bao auth enable kubernetes

bao write auth/kubernetes/config \
  kubernetes_host="https://kubernetes.default.svc:443"
```

If OpenBao itself runs in the cluster, that is enough: it uses its own pod token and the built-in CA to perform the TokenReview. If OpenBao runs outside, `kubernetes_ca_cert` and `token_reviewer_jwt` must be set as well.

Then the **role** that maps a ServiceAccount to policies:

```
bao write auth/kubernetes/role/eso-demo \
  bound_service_account_names=demo \
  bound_service_account_namespaces=demo \
  token_policies=eso-demo \
  token_ttl=1h
```

Check:

```
/ $ bao read auth/kubernetes/role/eso-demo
Key                                Value
---                                -
alias_name_source                  serviceaccount_uid
bound_service_account_names        [demo]
bound_service_account_namespaces   [demo]
token_policies                     [eso-demo]
token_ttl                          1h
```

Remember: the role says *who* (SA demo in namespace demo) and *with which permissions* (policy eso-demo). What those permissions concretely allow is in the policy – that is the step after the secrets engine.

Database secrets engine: the connection

```
bao secrets enable database
```

A **connection** describes how OpenBao connects to the database and – crucially – which roles may use this connection:

```

bao write database/config/postgres-demo \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@postgres-postgresql.postgres.svc.
    cluster.local:5432/demodb?sslmode=disable" \
  username="openbao" \
  password="<password of the PG role openbao>" \
  password_authentication="scram-sha-256" \
  allowed_roles="demo-app"

```

The PG role openbao must exist beforehand (Part III). The placeholders `{{username}}` and `{{password}}` are filled in by OpenBao itself; the advantage is that OpenBao can later rotate the password with `bao write -f database/rotate-root/postgres-demo` without anybody else ever knowing it.

`allowed_roles` is an allowlist. Only the database roles named here may create credentials through this connection. It is the first of two gates that later got in the way when creating the Miniflux role.

This is what the connection looks like after setup (OpenBao never shows the password again):

```

/ $ bao read database/config/postgres-demo
Key                               Value
---                               -
allowed_roles                     [demo-app]
connection_details                map[connection_url:postgresql://{{username}}:{{password}}@postgres-
postgresql.postgres.svc.cluster.local:5432/demodb?sslmode=disable password_authentication:
scram-sha-256 username:openbao]
plugin_name                       postgresql-database-plugin

```

Database secrets engine: the roles

A **role** in the database engine is a template: which SQL statements are executed when someone requests credentials, and how long do they live?

The first role: demo-app (read and write only)

This role already existed for another demo app. It is a good example of *least privilege* on existing data:

```

bao write database/roles/demo-app \
  db_name=postgres-demo \
  default_ttl=1h \
  max_ttl=24h \
  creation_statements='
    CREATE ROLE "{{name}}" WITH LOGIN PASSWORD ''''{{password}}'''' VALID UNTIL ''''{{e
      xpiration}}'''';
    GRANT CONNECT ON DATABASE demodb TO "{{name}}";
    GRANT USAGE ON SCHEMA public TO "{{name}}";
    GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO "{{name}}";
    GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO "{{name}}";'

```

(The `''''` construction is the POSIX way of placing a single quote inside a single-quoted string. Alternatively, write the statements to a file and pass `creation_statements=@file.sql`.)

OpenBao fills in the placeholders `{{name}}`, `{{password}}` and `{{expiration}}` on every call. `{{name}}` follows the pattern `v-<auth>-<role>-<random>-<ts>`, e.g. `v-kubernetes-demo-app-A1EdiNgGWCikBp74UKVD-1789630487`. The second part reveals which auth method requested the credentials – `kubernetes` for Kubernetes auth, `root` for the root token. That turns out to be surprisingly useful when debugging.

Why this role is no good for Miniflux – and this is the central lesson of the guide:

1. `CREATE` on the schema is missing. Since PostgreSQL 15 the pseudo-role `PUBLIC` no longer has `CREATE` on `public` by default. So Miniflux cannot create tables.
2. `GRANT ... ON ALL TABLES IN SCHEMA public` is a **snapshot**. It only affects tables that exist at the moment of the grant. On an empty database it grants on nothing. And the tables Miniflux then creates belong to the dynamic user – after an hour it is gone and its successor has no permissions on them.

The second role: miniflux (owner-role pattern)

The solution: a **fixed, non-login role** owns the schema and all objects. Every dynamic user becomes a member of that role and slips into it automatically on every session.

```
bao write database/roles/miniflux \  
  db_name=postgres-demo \  
  default_ttl=1h \  
  max_ttl=24h \  
  creation_statements='CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '""'{{password}}'""' V  
    ALID UNTIL '""'{{expiration}}'""' IN ROLE miniflux; ALTER ROLE "{{name}}" SET R  
    OLE miniflux;'
```

Two statements, two jobs:

- `... IN ROLE miniflux` makes the new user a member of the role `miniflux`. It *may* do everything `miniflux` may do.
- `ALTER ROLE "{{name}}" SET ROLE miniflux` sets the default role for every session of that user. Without this line the application would have to issue `SET ROLE miniflux` itself – Miniflux does not, and neither do most applications. With this line every connection *is* `miniflux` from the start, and everything it creates belongs to `miniflux`.

Side effect: because the dynamic user owns nothing itself, the `DROP ROLE` during revocation always succeeds. With the naive approach it fails as soon as the user has created tables (role cannot be dropped because some objects depend on it).

Extending the connection with the new role

This was the first error that came up:

```
/ $ bao read database/creds/miniflux  
Error reading database/creds/miniflux: ... Code: 500. Errors:  
* "miniflux" is not an allowed role
```

The connection only knows demo-app. A write to the config is a *merge* – existing fields are kept, only the fields passed are replaced (OpenBao briefly verifies the DB connection while doing so):

```
bao write database/config/postgres-demo allowed_roles="demo-app,miniflux"
```

For a demo, `allowed_roles="*"` would be more convenient. The explicit list is the cleaner way: it prevents anyone with write access to `database/roles/*` from obtaining credentials through a connection that is not theirs.

Policy: what may the logged-in token do?

The policy `eso-demo` is deliberately explicit – exactly one capability per path:

```
path "secret/data/demo/demo-app" {
  capabilities = ["read"]
}

path "secret/metadata/demo/demo-app" {
  capabilities = ["read"]
}

path "database/creds/demo-app" {
  capabilities = ["read"]
}

path "database/creds/miniflux" {
  capabilities = ["read"]
}
```

`bao policy write` replaces the policy entirely, so always pass the whole text:

```
bao policy write eso-demo - <<'EOF'
path "secret/data/demo/demo-app"      { capabilities = ["read"] }
path "secret/metadata/demo/demo-app"  { capabilities = ["read"] }
path "database/creds/demo-app"        { capabilities = ["read"] }
path "database/creds/miniflux"        { capabilities = ["read"] }
EOF
```

Policies are evaluated on *every request*, not at login. ESO tokens that have already been issued benefit immediately; no ESO restart is required.

That names the two gates that both have to pass before credentials flow:

Gate	Where	Error message when it is missing
allowed_roles	database/config/<connection>	"miniflux" is not an allowed role (HTTP 500)
Policy path	sys/policies/acl/<policy>	permission denied (HTTP 403)

Part III – Preparing PostgreSQL

All statements as superuser postgres in demodb (see Part I for the psql entry point).

The role OpenBao works with

OpenBao needs its own login user that may create and drop other roles. Not a superuser – CREATEROLE is enough:

```
CREATE ROLE openbao WITH LOGIN PASSWORD '...' CREATEROLE;
```

This password goes into database/config/postgres-demo once (Part II) and can be rotated by OpenBao afterwards.

The owner role for Miniflux

```
CREATE ROLE miniflux NOLOGIN;  
GRANT CONNECT ON DATABASE demodb TO miniflux;  
GRANT USAGE, CREATE ON SCHEMA public TO miniflux;  
GRANT miniflux TO openbao WITH ADMIN OPTION;
```

The last line has been mandatory since **PostgreSQL 16** and is easy to miss: CREATE ROLE ... IN ROLE miniflux is internally a GRANT miniflux TO <new user>, and only someone who holds ADMIN OPTION on miniflux may do that. Since PG 16, CREATEROLE alone is no longer sufficient. Without this line, bao read database/creds/miniflux fails with permission denied to grant role "miniflux".

Check with \du:

Role name	Attributes
demo	Create DB
miniflux	Cannot login
openbao	Create role
postgres	Superuser, Create role, Create DB,
...	
v-kubernetes-demo-app-A1EdiNgGWCikBp74UKVD-1789630487	Password valid until 2026-09-17 08:34:52+00
v-kubernetes-demo-app-DmrcWpu6p4vD4TULtYCp-1789632287	Password valid until 2026-09-17 09:04:52+00

The v-kubernetes-... entries are the dynamic users of the other demo app. Several existing at once is normal: ESO fetches new ones every 30 minutes, OpenBao only drops them once the lease expires (1 h). Two or three parallel users per role is the expected state.

Verification: does the session really run as miniflux?

Before touching anything in Kubernetes, a manual test pays off. Fetch credentials (with the root token; that is why the user is called v-root-...):

```
/ $ bao read database/creds/miniflux
Key          Value
----          -
lease_id     database/creds/miniflux/yVVxUMH7xmpld2lCx0kldNxq
lease_duration 1h
username     v-root-miniflux-WPF9Sj2Qy81wRslpzTRv-1789633845
password     <...>
```

And log in with them:

```
kubectl exec -i -n postgres postgres-postgresql-0 -- sh -c \
'PGPASSWORD="<pw>" psql -U v-root-miniflux-WPF9Sj2Qy81wRslpzTRv-1789633845 -d demodb' <
<'EOF'
SELECT current_user, current_role;
CREATE TABLE _probe(x int);
SELECT tableowner FROM pg_tables WHERE tablename = '_probe';
DROP TABLE _probe;
EOF
```

```
current_user | current_role
-----+-----
miniflux    | miniflux

CREATE TABLE
tableowner
-----
miniflux

DROP TABLE
```

Exactly what we wanted to see: the session is miniflux, and a newly created table belongs to miniflux – not to the v-root-... user. If current_role showed the v-... name here, ALTER ROLE ... SET ROLE would not have taken effect.

Part IV – Connecting the External Secrets Operator

Three objects, all in namespace demo.

SecretStore: how ESO reaches OpenBao

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: openbao
  namespace: demo
spec:
  provider:
    vault:
      server: http://openbao.openbao.svc:8200
      path: secret
      version: v2
      auth:
        kubernetes:
          mountPath: kubernetes
          role: eso-demo
          serviceAccountRef:
            name: demo
```

The SecretStore is needed for the KV part (secret/...). For dynamic credentials we do not use it directly – the generator brings its own provider configuration. The two look very similar, and for the same reason: both describe *how* and as *whom* ESO logs in.

VaultDynamicSecret: the generator

```
apiVersion: generators.external-secrets.io/v1alpha1
kind: VaultDynamicSecret
metadata:
  name: postgres-demo-creds
  namespace: demo
spec:
  path: /database/creds/miniflux
  method: GET
  resultType: Data

  provider:
    server: "http://openbao.openbao.svc:8200"
    auth:
      kubernetes:
        mountPath: kubernetes
        role: eso-demo
        serviceAccountRef:
          name: demo
```

path is the OpenBao path, method: GET corresponds to bao read, and resultType: Data tells ESO that the fields under .data (i.e. username and password) are the result. On every sync ESO calls this path anew – each time a new database user is created.

ExternalSecret: the target secret

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: miniflux-db
  namespace: demo
spec:
  refreshInterval: 30m

  target:
    name: miniflux-db
    creationPolicy: Owner
    template:
      data:
        DATABASE_URL: >--
          postgres://{{ .username }}:{{ .password }}@postgres-postgresql.postgres.svc.clus
          ter.local:5432/demodb?sslmode=disable

  dataFrom:
    - sourceRef:
        generatorRef:
          apiVersion: generators.external-secrets.io/v1alpha1
          kind: VaultDynamicSecret
          name: postgres-demo-creds
```

Two details:

- dataFrom with generatorRef instead of the usual secretStoreRef. That is the coupling to the generator.
- The template builds a complete DATABASE_URL from username and password. Miniflux expects exactly this one variable, so the Deployment stays free of any knowledge of OpenBao.

refreshInterval: 30m against default_ttl: 1h in OpenBao means: there are always valid credentials in the Secret before the old ones expire. Half the TTL is a good rule of thumb.

Apply and check

```
kubectl apply -f miniflux/vault-dynamic-secret.yaml
kubectl apply -f miniflux/miniflux-external-secret.yaml
kubectl get externalsecret -n demo miniflux-db
```

NAME	REFRESH INTERVAL	STATUS	READY	LAST SYNC
miniflux-db	30m	SecretSynced	True	8s

To force an immediate sync (e.g. after changing the generator), any change to an annotation on the ExternalSecret will do:

```
kubectl annotate externalsecret -n demo miniflux-db force-sync="$(date +%s)" --overwrite
```

And a look at the generated Secret, with the password masked:

```
kubectl get secret -n demo miniflux-db -o jsonpath='{.data.DATABASE_URL}' \
| base64 -d | sed -E 's#://([^:]+):[^\@]+\@#://\1:<pw>\@#'
```

```
postgres://v-kubernetes-miniflux-SkwRAKNo7i4xy0uFFD63-1789634099:<pw>@postgres-postgresql.po
stgres.svc.cluster.local:5432/demodb?sslmode=disable
```

v-kubernetes-miniflux-... – the credentials came through Kubernetes auth and the role miniflux. The chain is complete.

Part V – The application

The Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: miniflux
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: miniflux
  template:
    metadata:
      labels:
        app: miniflux
    spec:
      containers:
        - name: miniflux
          image: miniflux/miniflux:2.3.3
          ports:
            - containerPort: 8080
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: miniflux-db
                  key: DATABASE_URL
            - name: RUN_MIGRATIONS
              value: "1"
            - name: CREATE_ADMIN
              value: "1"
            - name: ADMIN_USERNAME
              value: "admin"
            - name: ADMIN_PASSWORD
              value: "admin123"
```

Nothing about this is OpenBao-specific. The Deployment reads a perfectly ordinary Kubernetes Secret. That is the point: the application does not need to know anything about dynamic credentials.

A tip from practice: during setup it helps to start the container with command: ["sh", "-c", "sleep infinity"]. Then you can `kubectl exec` into it, check `echo $DATABASE_URL` and start the bundled binary (`/usr/bin/miniflux`) manually. Before real operation that block must come out again – otherwise only `sleep` runs, and the application never starts.

```
kubectl apply -f miniflux/miniflux-deployment.yaml
kubectl rollout status -n demo deploy/miniflux
kubectl logs -n demo deploy/miniflux
```

A successful start looks like this:

```
level=INFO msg="Running database migrations" current_version=0 latest_version=132
level=INFO msg="Created new admin user" username=admin user_id=1
level=INFO msg="Starting HTTP server" listen_address=0.0.0.0:8080
```

Looking at the UI

For a quick look, a port-forward is enough, no Service or Ingress needed:

```
kubectl port-forward -n demo deploy/miniflux 8080:8080
```

Then open <http://localhost:8080> in the browser, log in with admin / admin123.

For permanent access: a Service of type ClusterIP plus an Ingress on Traefik, following the same pattern as the OpenBao chart.

Part VI – What went wrong, and why

This part is the real documentation. Every one of these errors was a gate that exists on purpose.

permission denied for schema public

```
level=INFO msg="Running database migrations" current_version=0 latest_version=132  
[Migration v1] pq: permission denied for schema public at position 2:17 (42501)
```

The pod was in `CrashLoopBackOff`. The connection itself worked – host, port, auth, database were all correct. The user simply was not allowed to create tables. Cause: PostgreSQL ≥ 15 no longer gives `PUBLIC CREATE` on the schema `public`, and the role `demo-app` did not grant it either.

Solution: owner role with `CREATE` on the schema, dynamic users become members (Part III).

"miniflux" is not an allowed role

HTTP 500 when reading `database/creds/miniflux`. The role existed but was not in the connection's `allowed_roles`. Solution: extend the allowlist (Part II).

permission denied (403) on everything

Two different causes with the same message:

1. **No token, or an expired one, in the calling terminal.** Recognisable by `bao token lookup` failing as well. Solution: `bao login`.
2. **The policy does not allow the path.** Recognisable with `bao token capabilities <path>` – it returns deny. Solution: extend the policy (Part II).

When debugging ESO the second cause is the more common one. The message then appears in the status of the `ExternalSecret` or in the logs of the ESO pod, not in the application.

permission denied to grant role "miniflux"

Did *not* occur here, because the grant with `ADMIN OPTION` had been set beforehand – but it is the typical trap from PostgreSQL 16 onwards with the owner-role pattern (Part III).

Logs of the wrong pod

During a rollout, `kubectl logs deploy/miniflux` likes to pick the old, terminating pod (Found 2 pods, using pod/...). When in doubt, name the pod explicitly or wait for `kubectl rollout status`.

Part VII – Operation

The TTL trap: environment variables are static

This is the most important open issue in the current setup. ESO renews the Secret every 30 minutes, but a container reads environment variables **only at startup**. After an hour OpenBao drops the user Miniflux was started with. Existing connections in the pool keep working, but every new connection attempt fails – the application degrades quietly.

Three ways out:

- **Restart trigger on Secret change.** Stakater's *Reloader* watches Secrets and restarts affected Deployments. One annotation on the Deployment is enough: `reloader.stakater.com/auto: "true"`. A restart every 30 minutes is unproblematic for Miniflux.
- **Mount the Secret as a file** instead of an environment variable. Kubernetes updates mounted Secrets inside the running container. But that only helps if the application re-reads the file on every connection attempt – Miniflux does not.
- **Longer TTL.** `default_ttl=24h` with `refreshInterval: 12h` reduces the frequency but does not solve the problem.

For production the first option is the usual route.

Retiring the root token

The setup ran with the root token. After the initial bootstrap it should be revoked (`bao token revoke <root-token>`) and, if needed, regenerated from the unseal keys (`bao operator generate-root`). For day-to-day administration: a dedicated policy and e.g. userpass or OIDC login.

Infrastructure as Code

All bao commands from Part II are state that is lost on a rebuild. The repository already manages auth methods, policies and audit with OpenTofu under `tofu/`. The objects shown here belong there:

Object	Tofu resource
Database engine	<code>vault_mount (type = "database")</code>
Connection postgres-demo	<code>vault_database_secret_backend_connection</code>
Roles demo-app, miniflux	<code>vault_database_secret_backend_role</code>
Policy eso-demo	<code>vault_policy</code>
K8s auth role eso-demo	<code>vault_kubernetes_auth_backend_role</code>

The SQL statements from Part III (roles openbao, miniflux) are database state and belong in an init script or a migration of the Postgres deployment.

Plain-text passwords

ADMIN_PASSWORD: "admin123" in the Deployment and all passwords in postgres/values.yaml are demo compromises. The Miniflux admin password belongs in the KV engine (secret/demo/miniflux) and into the Deployment via ExternalSecret – just like DATABASE_URL, only without a generator.

Lease hygiene

Whoever tests a lot produces a lot of leases. Overview and cleanup:

```
bao list sys/leases/lookup/database/creds/miniflux
bao lease revoke -prefix database/creds/miniflux    # all at once
```

The revoke executes the DROP ROLE in PostgreSQL. Thanks to the owner-role pattern it always works, because the dynamic users own no objects.

Appendix A – All commands at a glance

In the order they are needed. Prerequisites: cluster, Helm, kubectl context.

```

# — Cluster —————
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao --create-namespace -f helm/values.yaml
kubectl exec -n openbao openbao-0 -- bao operator init
kubectl exec -n openbao openbao-0 -- bao operator unseal # x3

helm upgrade --install postgres oci://registry-1.docker.io/bitnamicharts/postgresql \
  -n postgres --create-namespace -f postgres/values.yaml

helm upgrade --install external-secrets external-secrets/external-secrets \
  --version 2.10.0 -n external-secrets --create-namespace

kubectl create namespace demo
kubectl create serviceaccount demo -n demo

# — PostgreSQL (as postgres in demodb) —————
CREATE ROLE openbao WITH LOGIN PASSWORD '...' CREATEROLE;
CREATE ROLE miniflux NOLOGIN;
GRANT CONNECT ON DATABASE demodb TO miniflux;
GRANT USAGE, CREATE ON SCHEMA public TO miniflux;
GRANT miniflux TO openbao WITH ADMIN OPTION;

# — OpenBao (root token, shell in the pod) —————
bao auth enable kubernetes
bao write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
bao write auth/kubernetes/role/eso-demo \
  bound_service_account_names=demo bound_service_account_namespaces=demo \
  token_policies=eso-demo token_ttl=1h

bao secrets enable database
bao write database/config/postgres-demo \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@postgres-postgresql.postgres.svc.
    cluster.local:5432/demodb?sslmode=disable" \
  username=openbao password='...' password_authentication=scram-sha-256 \
  allowed_roles="demo-app,miniflux"

bao write database/roles/miniflux db_name=postgres-demo default_ttl=1h max_ttl=24h \
  creation_statements='CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '""'{{password}}'""' V
    ALID UNTIL '""'{{expiration}}'""' IN ROLE miniflux; ALTER ROLE "{{name}}" SET R
    OLE miniflux;'

bao policy write eso-demo - <<'EOF'
path "secret/data/demo/demo-app" { capabilities = ["read"] }
path "secret/metadata/demo/demo-app" { capabilities = ["read"] }
path "database/creds/demo-app" { capabilities = ["read"] }
path "database/creds/miniflux" { capabilities = ["read"] }
EOF

bao read database/creds/miniflux # test

# — ESO + app —————
kubectl apply -f miniflux/vault-dynamic-secret.yaml
kubectl apply -f miniflux/miniflux-external-secret.yaml
kubectl apply -f miniflux/miniflux-deployment.yaml
kubectl get externalsecret -n demo miniflux-db
kubectl logs -n demo deploy/miniflux
kubectl port-forward -n demo deploy/miniflux 8080:8080

```

Appendix B – Glossary

Auth method – How a client proves its identity to OpenBao. Here: kubernetes (ServiceAccount token) and token (root token).

Policy – The rule set describing which paths a token may use with which capabilities (read, create, update, delete, list). Default: everything denied.

Secrets engine – A plugin that provides secrets. kv stores static values, database generates dynamic credentials.

Connection – In the database engine: the link to one concrete database including the allowlist of roles (allowed_roles).

Role (OpenBao) – In the database engine: the SQL template for creating a user, plus TTL. Not to be confused with a PostgreSQL role.

Role (PostgreSQL) – A user or a group. LOGIN makes it a user, NOLOGIN a pure group. Membership via GRANT role TO user or CREATE ROLE ... IN ROLE role.

Lease – The lifetime of a dynamic secret. When it expires, OpenBao runs the revocation (here: DROP ROLE).

ADMIN OPTION – Allows a role to make others members of a role. Since PostgreSQL 16 a prerequisite for IN ROLE by non-superusers.

SET ROLE – Switches the effective role within a session. ALTER ROLE u SET ROLE r makes that the default when connecting.

Generator (ESO) – An ESO object that *creates* a new secret on every sync instead of reading one. VaultDynamicSecret calls an OpenBao path for that.

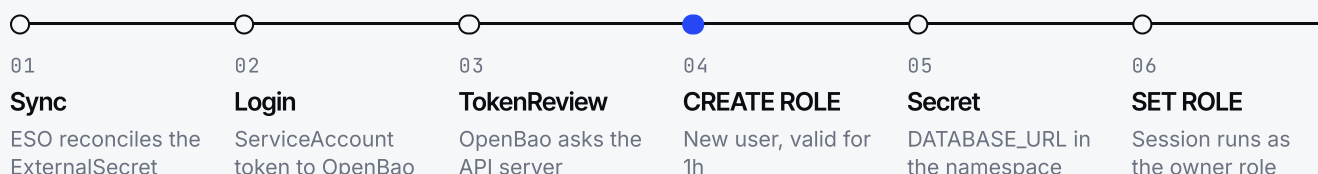
TokenReview – The Kubernetes API OpenBao uses to check that a ServiceAccount token is genuine and valid.

The application no longer gets a fixed password. It gets a user that will not exist an hour from now.

This guide documents a complete, actually executed build: OpenBao creates short-lived PostgreSQL users on demand, the External Secrets Operator writes them into a Kubernetes Secret, and Miniflux reads them like any other environment variable – without ever knowing about OpenBao.

Every command, every manifest, every SQL statement. And every error that happened along the way: permission denied for schema public, is not an allowed role, the 403 that was just a missing login. Each one is a gate that exists on purpose.

```
-- The core of the guide in two lines
CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}'
  VALID UNTIL '{{expiration}}' IN ROLE miniflux;
ALTER ROLE "{{name}}" SET ROLE miniflux;
```



kubernetes	RKE2 · Traefik · Longhorn
openbao	2.6.2 · chart 0.29.4 · raft
postgresql	18.6 · bitnami chart 18.11.3
external-secrets	2.10.0 · VaultDynamicSecret
miniflux	2.3.3

28 pages · 7 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty